

Recommender System for movie ratings

Methods of classification and dimensionality reduction - Project 1

Tomasz Kąkol, Mikołaj Płóciniczak

Contents

Introduction	2
Objective and Data	2
Models	2
Imputation methods	3
Non-negative Matrix Factorization (NMF) algorithm	3
Overview of the NMF algorithm	3
Performance of the NMF algorithm	4
Singular Value Decomposition (SVD) methods	4
SVD1 - Non-iterative version	4
Overview of the SVD1 algorithm	4
Performance of the SVD1 algorithm	5
SVD2 - Iterative version	5
Overview of the SVD2 algorithm	6
Performance of the SVD2 algorithm	6
Stochastic Gradient Descent (SGD) algorithm	7
Overview of the SGD algorithm	7
Performance of the SGD algorithm	7
Model comparison	8
Summary	9
Appendix: Implementation	9

Introduction

Recommender systems play a crucial role in modern online platforms by helping users discover relevant content based on their preferences. In this project, we focus on building a movie recommendation system that predicts user ratings for unseen user–movie pairs. The task is based on the MovieLens dataset, which contains historical ratings provided by users for various movies. The main challenge in this problem is the sparsity of the data, as most users have rated only a small subset of available movies. To address this issue, we apply several matrix factorization techniques, including NMF, SVD-based methods, and optimization-based approaches such as Stochastic Gradient Descent. These methods aim to learn latent representations of users and movies, enabling accurate prediction of missing ratings. The performance of the implemented models is evaluated using the Root Mean Square Error (RMSE), allowing us to compare different approaches and identify the most effective solution.

Objective and Data

The objective of this project is to design and implement a movie recommender system that can accurately predict user preferences based on historical rating data¹. The system aims to learn patterns in how users rate movies and use this knowledge to estimate ratings for user–movie pairs that have not yet been observed. The dataset used for this task consists of user interaction data in the form of tuples containing a user identifier, a movie identifier, and a numerical rating (ranging from 0 to 5, including half-point values). This type of data is inherently sparse, as each user rates only a small fraction of the available movies, resulting in many missing entries.

To facilitate modeling, the data is transformed into a matrix representation, where rows correspond to users and columns correspond to movies, and each entry represents a rating if available, or is missing otherwise. The core idea behind the recommender system is to approximate this sparse matrix using various techniques, such as matrix factorization methods (NMF, SVD and SGD). These methods decompose the original matrix into lower-dimensional representations that capture latent features of users and movies. By reconstructing the matrix from these learned representations, the system can infer missing values and thus predict how a user is likely to rate a given movie.

Models

In this section, we focus on presenting the details of the algorithms used to approximate the original data, which in turn enables us to generate predictions. To evaluate the performance of each algorithm, we use the Root Mean Square Error (RMSE) metric following a standard procedure. Before training a given model, the dataset is split into a training set and a test set (in an 80%–20% proportion²). The model is then trained on the training set, and its performance is evaluated on the test set using the RMSE defined as:

$$\text{RMSE} = \sqrt{\frac{1}{|\mathcal{T}|} \sum_{(u,v) \in \mathcal{T}} (\mathbf{Z}'[u,m] - \mathbf{T}[u,m])^2}$$

where $\mathcal{T}$ denotes the test set, $\mathbf{Z}'[u,m]$ is the predicted rating for user u and movie m , and $\mathbf{T}[u,m]$ is the corresponding true rating from the test set.

¹<https://grouplens.org/datasets/movielens/>

²To ensure fairness we split the data once and train all models using the same split.

Imputation methods

Before presenting the details of the individual algorithms, we first discuss the methods used to handle missing entries in the original data matrix, as most of the approaches described below (with the exception of SGD) rely on such preprocessing. As mentioned earlier, one of the main challenges in building a recommender system for this type of data is its sparsity—only a small fraction of all possible (user, movie) pairs is observed. At the same time, many matrix approximation techniques require a fully populated input matrix.

To address this issue, we consider three different strategies for imputing missing values (with abbreviated names given in parentheses):

- (i) replacing missing entries with 0 (Zero),
- (ii) replacing missing entries with the average rating of a given movie (Movie Mean),
- (iii) replacing missing entries with the average rating of a given user (User Mean).

As will be shown later, the choice of imputation method has a significant impact on the performance of the resulting algorithms.

Non-negative Matrix Factorization (NMF) algorithm

In this section we focus on the Non-negative Matrix Factorization (NMF) algorithm - we start by going through the technical details of the algorithm and later we show the evaluated RMSE score for different imputation methods and parameter values.

Overview of the NMF algorithm

Let $\mathbf{Z}$ be an $n \times d$ matrix (where n and d denote the number of users and movies, respectively) corresponding to the original data, after applying the chosen imputation method. Our goal is to approximate $\mathbf{Z}$ as a product of two lower-dimensional matrices:

$$\mathbf{Z} \approx \mathbf{Z}' = \mathbf{W}\mathbf{H}$$

where $\mathbf{W} \in \mathbb{R}^{n \times r}$ and $\mathbf{H} \in \mathbb{R}^{r \times d}$, with r being the number of latent factors. To obtain a good approximation, we seek matrices $\mathbf{W}$ and $\mathbf{H}$ that minimize the reconstruction error, defined as:

$$\underset{\mathbf{W}, \mathbf{H}}{\operatorname{argmin}} = \sum_{i,j} (z_{ij} - w_i^T h_j)^2$$

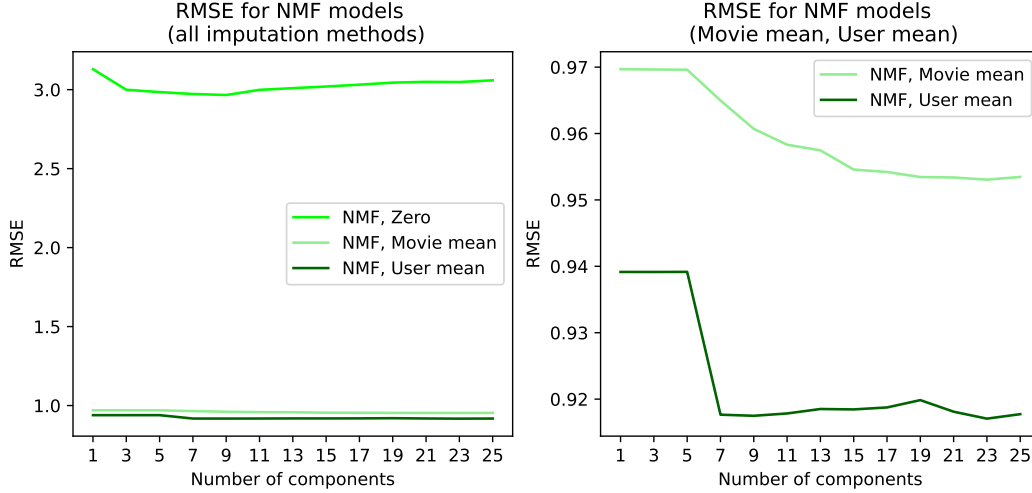
subject to the non-negativity constraints:

$$\mathbf{W} \geq 0, \quad \mathbf{H} \geq 0$$

The matrices are computed using a coordinate descent approach combined with projection onto the non-negative orthant to enforce the constraints. The algorithm is terminated either when the maximum number of iterations is reached or when the change in the loss function falls below a predefined tolerance threshold.

Performance of the NMF algorithm

As described in the previous subsection, the NMF algorithm is parameterized by the value of r , which determines the dimensionality of the factor matrices used to approximate the original data matrix. The choice of this parameter, as well as the selected imputation strategy, has a direct impact on the quality of the resulting predictions. Below, we present the RMSE values obtained for each of the three imputation methods and for various values of the parameter r .



As can be observed, the dominant factor influencing model performance is the choice of imputation strategy. In particular, the Zero imputation method—where missing entries are replaced with zeros—yields the worst RMSE results. The remaining two approaches, Movie Mean and User Mean, perform significantly better and achieve comparable results overall, with the User Mean method providing the best performance, as illustrated in the second figure. Additionally, the second plot shows that for values of r greater than 15, the RMSE stabilizes and does not improve significantly with further increases in model complexity. As will be seen in the analysis of subsequent algorithms, the observed impact of the imputation strategy remains consistent across different methods.

Singular Value Decomposition (SVD) methods

In this section we focus on two algorithms using Singular Value Decomposition (SVD). We start by going through the technical details of the algorithms and later we show the evaluated RMSE score for different imputation methods and parameter values for both of these algorithms.

SVD1 - Non-iterative version

In this part we focus on the simpler, non-iterative version of the two models using the SVD method. We start by presenting details regarding the algorithm.

Overview of the SVD1 algorithm

The core idea of this method is based on the theorem on approximating a rectangular matrix using its Singular Value Decomposition (SVD). Let us again assume that $\mathbf{Z}$ denotes the original data matrix, after applying the chosen imputation strategy. From linear algebra, we know that any such matrix can be decomposed into the product of two orthogonal matrices and a diagonal matrix containing non-negative singular values:

$$\mathbf{Z} = \mathbf{U}\mathbf{A}\mathbf{V}^T$$

To obtain a low-rank approximation, we consider truncated matrices $\mathbf{U}_r$ and $\mathbf{V}_r$, formed by taking the first r columns of $\mathbf{U}$ and $\mathbf{V}$, respectively, as well as $\mathbf{\Lambda}_r$, which is the $r \times r$ principal submatrix of $\mathbf{\Lambda}$. This leads to the following approximation:

$$\mathbf{Z} \approx \mathbf{Z}' = \mathbf{U}_r \mathbf{\Lambda}_r \mathbf{V}_r^T$$

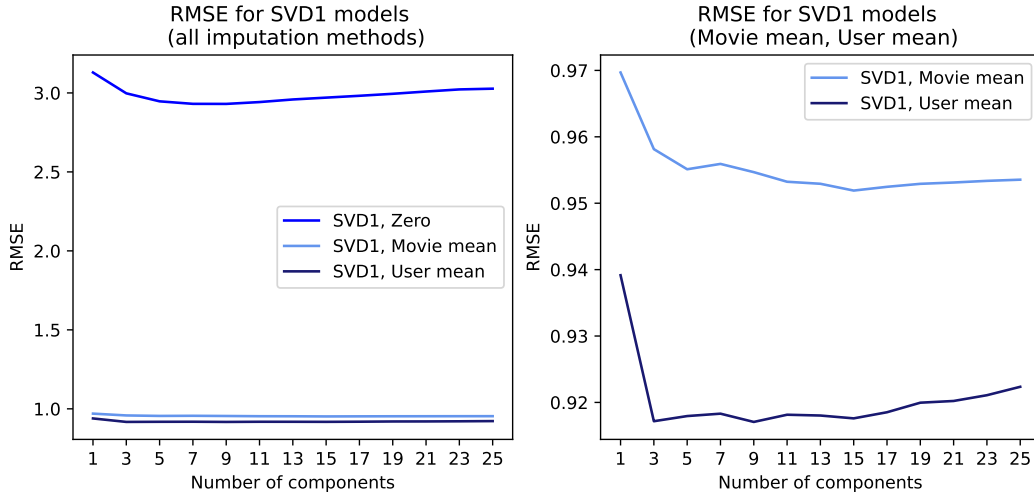
This expression can be further rewritten as:

$$\mathbf{Z}' = \mathbf{U}_r \sqrt{\mathbf{\Lambda}_r} \sqrt{\mathbf{\Lambda}_r} \mathbf{V}_r^T = \mathbf{W}_r \mathbf{H}_r$$

where $\mathbf{W}_r = \mathbf{U}_r \sqrt{\mathbf{\Lambda}_r}$ and $\mathbf{H}_r = \sqrt{\mathbf{\Lambda}_r} \mathbf{V}_r^T$. Thus, the approximation $\mathbf{Z}'$ can again be expressed as the product of two lower-dimensional matrices, making this approach conceptually similar to the previously described NMF method.

Performance of the SVD1 algorithm

Similarly to the NMF algorithm, the currently discussed SVD-based method is parameterized by the value of r , which determines the dimensionality of the matrices used to approximate the original data matrix. The choice of this parameter, together with the selected imputation strategy, has a significant impact on the quality of the resulting predictions. Below, we present the RMSE values obtained for different values of r and for each of the considered data imputation methods.



As shown in the plots above, replacing missing values with zeros once again leads to the worst performance among all considered imputation strategies. The Movie Mean and User Mean approaches produce substantially better and relatively similar results, with the User Mean method maintaining a noticeable advantage. As in the case of the NMF method, the RMSE values stabilize as r increases, indicating that further increases in model complexity do not yield meaningful improvements in predictive accuracy. However, unlike NMF, where stabilization occurred for values of r greater than 15, here the performance stabilizes already at $r = 3$, suggesting that this method achieves strong predictive performance even with relatively low model complexity.

SVD2 - Iterative version

In this part we focus on the second, iterative version of the two models using the SVD method. We start by presenting details regarding the algorithm.

Overview of the SVD2 algorithm

We follow an approach analogous to the previously discussed algorithm, in which the original data matrix is approximated by reducing the dimensionality of the matrices obtained from its SVD decomposition. Let $\mathbf{Z}$ denote the original data matrix (after imputation using a selected method). The first step of the algorithm is to perform Singular Value Decomposition (SVD) on $\mathbf{Z}$ and reduce the dimensionality of the resulting factors to rank r (we keep only the first r singular values and corresponding singular vectors):

$$\mathbf{Z}^{(0)} = \text{SVD}_r[\mathbf{Z}] = \mathbf{U}_r \mathbf{\Lambda}_r \mathbf{V}_r^T$$

Next, we iteratively refine the approximation by repeating the SVD procedure on an updated version of the matrix. In each iteration, we construct a new matrix where:

- observed (originally non-missing) entries are kept unchanged from $\mathbf{Z}$,
- missing entries are replaced with the corresponding values from the previous approximation $\mathbf{Z}^{(n)}$.

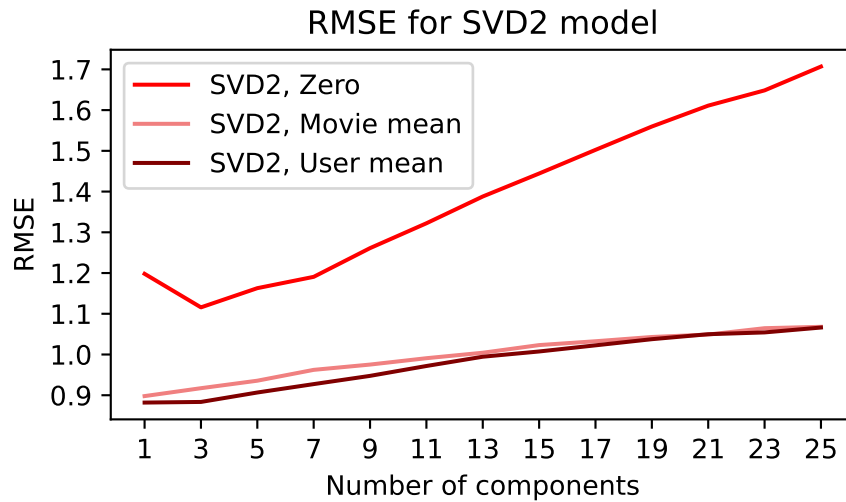
We then apply a rank- r SVD to this updated matrix:

$$\mathbf{Z}^{(n+1)} = \text{SVD}_r \left[\begin{array}{c} \mathbf{Z} \\ \mathbf{Z}^{(n)} \end{array} \begin{array}{l} \text{where } z_{ij} \neq '?' \\ \text{elsewhere} \end{array} \right]$$

We repeat this procedure until either a maximum number of iterations is reached or the change in the loss function between successive iterations falls below a predefined tolerance threshold.

Performance of the SVD2 algorithm

As in the two previously discussed algorithms, this method is parameterized by the value of r , which determines the rank used in the iterative SVD decompositions. The choice of this parameter, as well as the method used to initialize (impute) the original matrix, has an impact on the prediction performance. The figure below shows the RMSE values as a function of r for different imputation methods.



Similarly to the previous cases, the Zero imputation method yields the worst performance. However, the differences in RMSE between this method and the others are smaller than before. This is likely because the algorithm is iterative, meaning that the initial imputation has a weaker influence on the final result compared to the earlier methods. Once again, the User Mean imputation approach provides the best results. In contrast to the previous algorithms, increasing the value of r now leads to worse RMSE values. This behavior is likely due to overfitting, as well as the repeated reinforcement of erroneous predictions across successive SVD iterations.

Stochastic Gradient Descent (SGD) algorithm

In this section we focus on Stochastic Gradient Descent (SGD) algorithm - we start by going through the technical details of the algorithm and later we show the evaluated RMSE score for different parameter values.

Overview of the SGD algorithm

Once again, let $\mathbf{Z}$ denote the original data matrix. In this case, no prior imputation is required, since missing entries are naturally handled within the optimization process. As in the NMF model, the goal is to approximate $\mathbf{Z}$ as a product of two lower-dimensional matrices:

$$\mathbf{Z} \approx \mathbf{Z}' = \mathbf{W}\mathbf{H}$$

where $\mathbf{W} \in \mathbb{R}^{n \times r}$ and $\mathbf{H} \in \mathbb{R}^{r \times d}$, with r being the number of latent factors. To obtain a good approximation, we seek matrices $\mathbf{W}$ and $\mathbf{H}$ that minimize the reconstruction error computed only over observed entries:

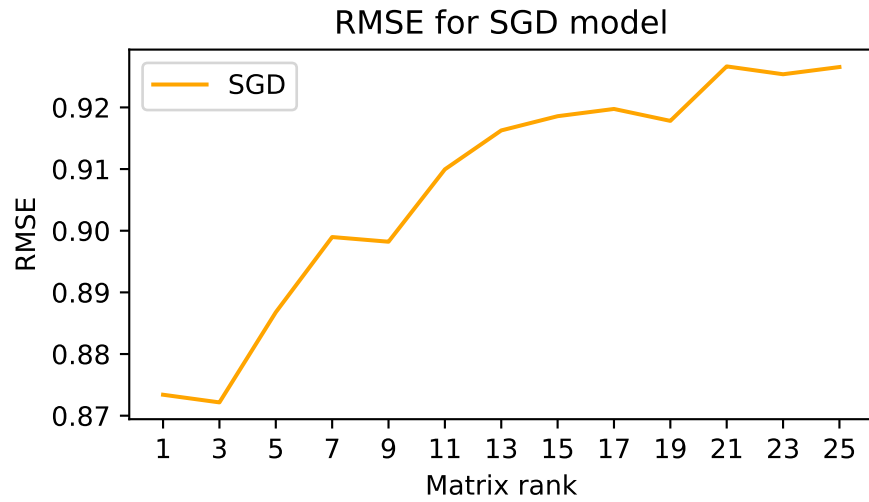
$$\underset{\mathbf{W}, \mathbf{H}}{\operatorname{argmin}} = \sum_{(i,j): z_{ij} \neq '?'} (z_{ij} - w_i^T h_j)^2$$

Unlike in the NMF setting, the loss function here explicitly ignores missing values, which eliminates the need for imputing the original matrix. In practice, the matrices $\mathbf{W}$ and $\mathbf{H}$ are learned using stochastic gradient-based optimization. In particular, we parameterize the model with user and item latent vectors (corresponding to rows of $\mathbf{W}$ and columns of $\mathbf{H}$), and additionally include bias terms for users and items, as well as a global mean. For a given pair (i, j) , the predicted value is computed as the sum of the global mean, user bias, item bias, and the dot product of the corresponding latent vectors.

The model is trained using mini-batch optimization with the Adam (Adaptive Moment Estimation) optimizer. At each iteration, a batch of observed entries is sampled, predictions are computed, and the mean squared error loss is evaluated. Gradients of this loss with respect to all model parameters (latent factors and biases) are then computed via backpropagation, and the parameters are updated using Adam, which adaptively adjusts learning rates based on first and second moments of the gradients. Additionally, L^2 regularization (weight decay) is applied to prevent overfitting. This procedure is repeated for a fixed number of epochs, and the final model parameters define the factorized approximation of the original matrix.

Performance of the SGD algorithm

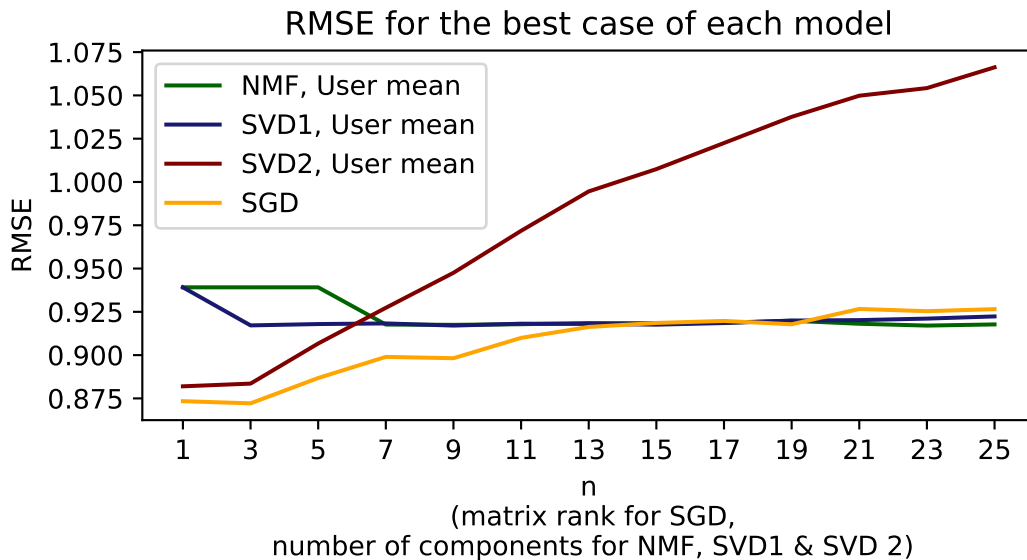
Similarly to the previously discussed algorithms, the currently discussed SGD method is parameterized by the value of r , which determines the dimensionality of the matrices used to approximate the original data matrix. The choice of this parameter, together with the selected imputation strategy, has a significant impact on the quality of the resulting predictions. Below, we present the RMSE values obtained for different values of r and for each of the considered data imputation methods.



As we can observe, similarly to the second SVD-based model, increasing the parameter r (which determines the dimensionality of the factor matrices used to approximate the original data matrix) does not improve the algorithm's performance. The lowest RMSE is achieved for $r = 3$. This behavior is most likely due to overfitting, as well as the propagation and reinforcement of erroneous predictions in the learned representations.

Model comparison

In this section, we compare the previously discussed models to determine which of them performs best in predicting unknown movie ratings, as measured by the RMSE metric. The figure below presents a comparative summary of RMSE results for each model, using the most effective data imputation strategy in each case.



The best overall performance is achieved by the SGD model, with the lowest RMSE observed at the parameter value equal to 3. Second, the SVD2 model yields the next best results, with performance closest to SGD, particularly for the parameter values also near 3. In contrast, the SVD1 and NMF models exhibit very similar behavior across all tested values of the model parameter, with relatively stable RMSE values that do not significantly depend on the choice of this parameter. Overall, the results suggest that lower values of r , corresponding to simpler (lower-complexity) models, tend to provide the best predictive performance.

Summary

We began by introducing the problem of movie rating prediction and formulating it in terms of a matrix representation. We then presented four different approaches to approximating the rating matrix: NMF, SVD1, SVD2, and SGD. These methods enable the prediction of previously unknown ratings. Each algorithm was evaluated using a standard train–test split procedure, with performance measured in terms of RMSE. For models that required imputing missing values, we found that the most effective strategy in all cases was to replace missing entries with the corresponding user mean. Within this group, the SVD2 model achieved the best results. Overall, the strongest predictive performance was obtained with the SGD-based model, which does not rely on any explicit imputation strategy.

Appendix: Implementation

The complete code is available at: <https://github.com/miplock/movie-recommender-ml>. Instructions for running the different modes (model training and prediction) are provided in the README file.